

QpiAI™ - Joint AI Training and Certification Program

Non-Linear Manifold Learning

Sourjya Sarkar

Manifold learning

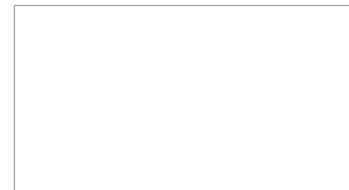
Recovering low dimensional data representation embedded within a higher dimensional space

Embeddings may be linear or non-linear

Learn a map

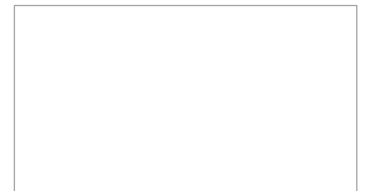
$$f: \mathcal{M} \rightarrow \mathbb{R}^n, \quad \mathcal{M} \subseteq \mathbb{R}^d$$

That preserves distances between points that are close by

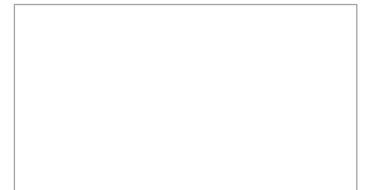


NonLinear Manifold Learning

- Kernel PCA (KPCA)
- Locally Linear Embedding (LLE)
- Laplacian EigenMaps (LE)
- ISOMAP
- t-Stochastic Neighbor Embedding (t-SNE)



- Stochastic Neighbor Embedding (SNE)
- t-SNE
- Applications

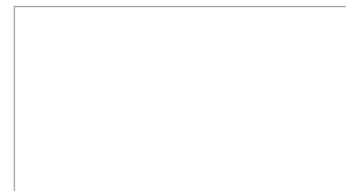


Stochastic Neighbor Embedding (SNE) Summary

- A **data point** – is a point x_i in the original **data space** $\mathbb{R}^d$
- A **map point** – is point y_i in the **map space** $\mathbb{R}^2/\mathbb{R}^3$. Every map point represents one of the original data points

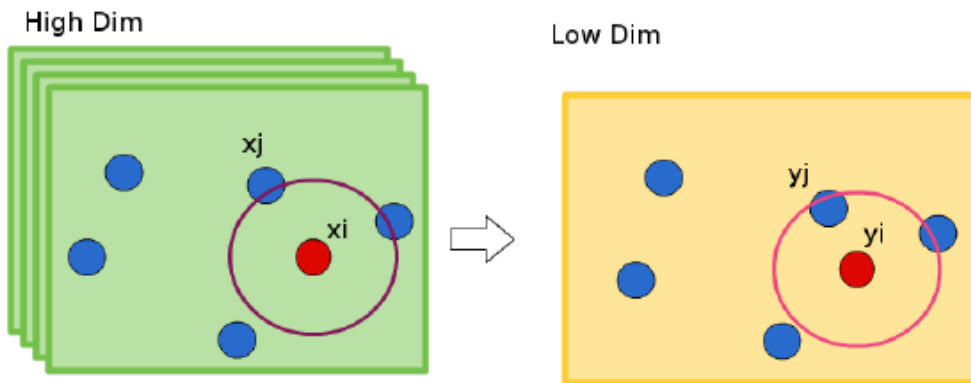
SNE procedure

- Compute an $N \times N$ similarity matrix (probabilistic distances) in the high-dimensional input space
- Define an $N \times N$ similarity matrix (probabilistic distances) in the low-dimensional embedding space
- Define cost function – sum of KL divergence between the two probability distributions at each point
- Iteratively learn low-dimensional embedding by minimizing the cost function using gradient descent



Neighborhood and Distance Preservation

Measure pairwise similarities between high-dimensional and low-dimensional objects



$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

Stochastic Neighbor Embedding (SNE)

Converting the high-dimensional Euclidean distances into conditional probabilities that represent similarities

- Similarity of datapoints in High Dimension

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

- Similarity of datapoints in Low Dimension

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}$$

- Cost function

$$C = \sum_i KL(P_i \| Q_i) = \sum_i \sum_j P_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

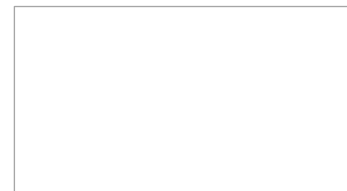
Minimize the cost function using gradient descent

Kullback Liebler (KL) divergence

Measures distance between two distributions, P and Q :

$$KL(P\|Q) = \sum_{ij} p_{ij} \log \left(\frac{p_{ij}}{q_{ij}} \right)$$

- Not symmetric!
- Code theory intuition : if we are transmitting information that is distributed according to P , then the optimal (lossless) compression will need to send an average $H(P) = - \sum_{ij} p_{ij} \log_2 p_{ij}$ bits.
- What happens if you expect Q (and design your compression accordingly) but the actual distribution is P ?
 - Will send an average of $H(Q) + KL(P\|Q)$ bits
 - $KL(P\|Q)$ is the “penalty” for using wrong distribution.

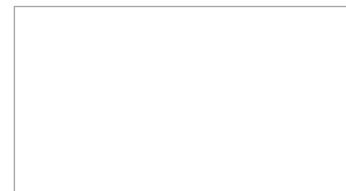


Gradient has a surprisingly simple form

$$\frac{\partial \mathcal{C}}{\partial y_i} = 2 \sum_{j \neq i} (p_{j/i} - q_{j/i} + p_{i/j} - q_{i/j}) (y_i - y_j)$$

The gradient update with momentum term is given by

$$Y^{(t)} = Y^{(t-1)} + \eta \frac{\partial \mathcal{C}}{\partial y_i} + \beta(t)(Y^{(t-1)} - Y^{(t-2)})$$



Minimize the sum of the KL divergences between the conditional probabilities

$$C = \sum_i KL(P_i \| Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

Minimize a single KL divergences between a joint probability distribution

$$C = KL(P \| Q) = \sum_i \sum_{j \neq i} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

The obvious way to redefine the pairwise similarities is

$$p_{ij} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma^2)}$$

$$q_{ij} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}$$

Such that $p_{ij} = p_{ji}$, $q_{ij} = q_{ji}$, the main advantage is simplifying the gradient

$$\frac{\partial \mathcal{C}}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)$$

However, in practice we symmetrize (or average) the conditionals

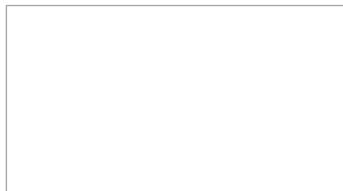
$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

Set the bandwidth σ_i such that the conditional has a fixed perplexity (effective number of neighbors)
 $Perp(P_i) = 2^{H(P_i)}$, typical value is about 5 to 50

where $H(P_i)$ is the Shannon Entropy of P_i measured in bits given by
 $H(P_i) = - \sum_j p_{ij} \log_2 p_{ij}$

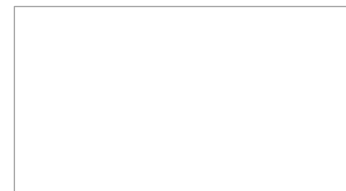
SNE Algorithm

- We have P , and are looking for $y^{(1)}, \dots, y^{(N)} \in \mathbb{R}^k$ such that the distribution Q we infer will minimize $L(Q) = KL(P\|Q)$
- Note that $KL(P\|Q) = \sum_{ij} p_{ij} \log\left(\frac{p_{ij}}{q_{ij}}\right) = -\sum_{ij} p_{ij} \log(q_{ij}) + \text{const}$
- Can show that $\frac{\partial L}{\partial y^{(i)}} = \sum_j (p_{ij} - q_{ij})(y^{(i)} - y^{(j)})$
- Main issue – **crowding problem**



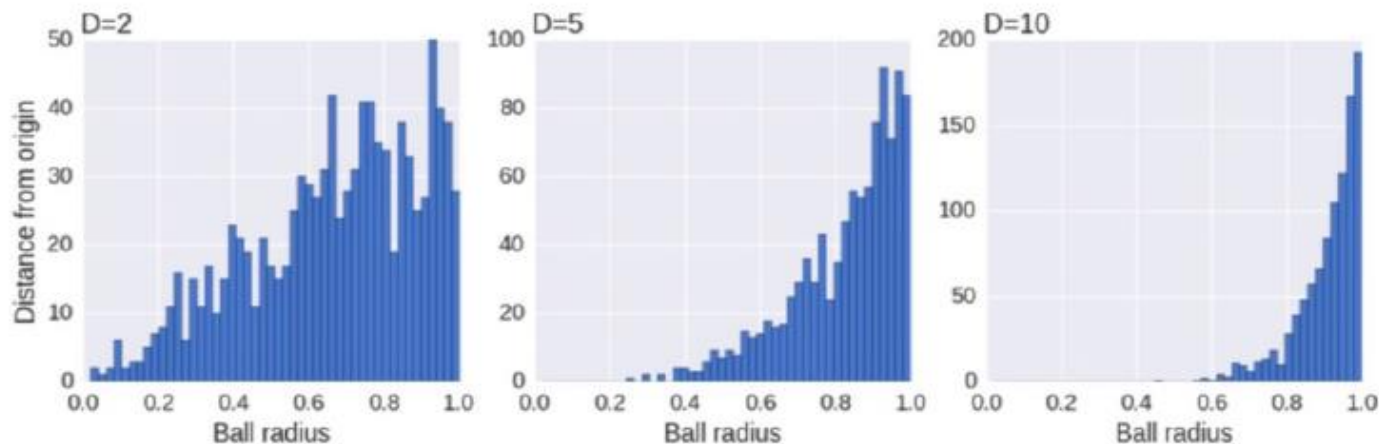
SNE Issues: Crowding Problem

- In high dimension we have more room, points can have a lot a different neighbors
- In 2D a point can have a few neighbors at distance one all far from each other – what happens when we embed in 1D?
- This is the “crowding problem” – we don’t have enough room to accommodate all neighbors.
- This is one of the biggest problems with SNE.
- **t-SNE solution** : Change the **Gaussian** in **Q** to a **heavy tailed distribution**.

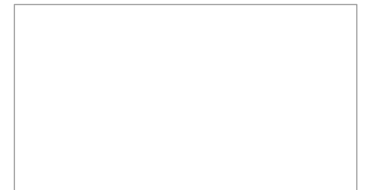


SNE Issues: Crowding Problem

SNE suffers from the “crowding problem”: The area of the 2D map that is available to accommodate moderately distant data points will not be large enough compared with the area available to accommodate nearby data point.



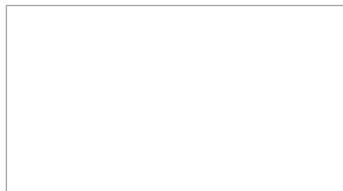
- Stochastic Neighbor Embedding (SNE)
- t-SNE
- Applications



- Student-t Probability density $p(x) \propto (1 + \frac{x^2}{v})^{-(v+1)/2}$
- For $v = 1$ (degree of freedom) we get $p(x) \propto \frac{1}{1+x^2}$
- Student-t is an infinite mixture of Gaussians with different variances
- Student-t is computationally cheaper than Gaussians (no exponent)
- Probability goes to zero slower than a Gaussian.
- We can now redefine q_{ij} as

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + \|y_k - y_l\|^2)^{-1}}$$

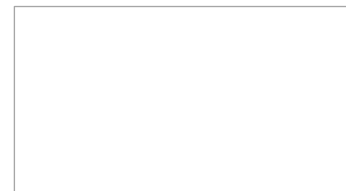
- We leave p_{ij} as is !



t-SNE: Probabilistic Distances

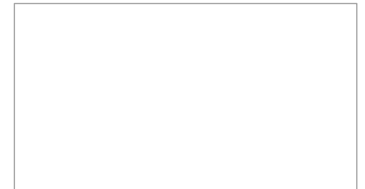
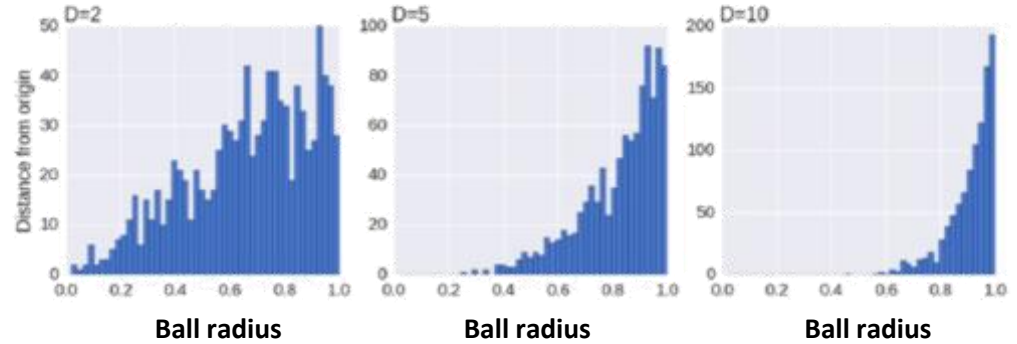
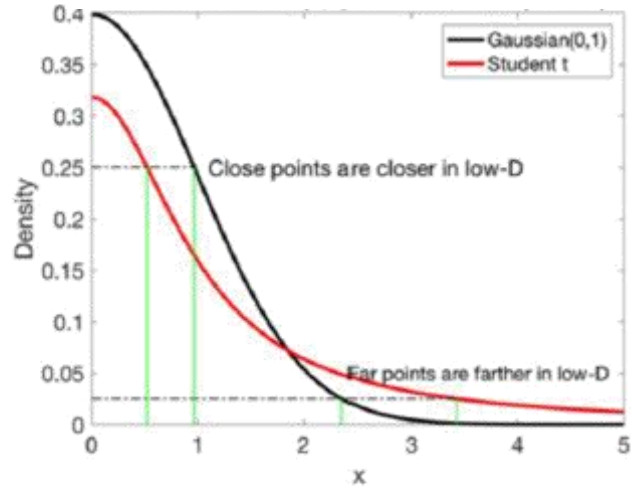
$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}$$

- Numerator term approaches an inverse square law for large pairwise distances in the low-dimensional map. This makes the map's representation of joint probabilities (almost) invariant to changes in the scale of the map for map points that are far apart.
- It also means that large cluster of points far apart interact like individual points.
- The heavy tails of the normalized Student-t distribution allow dissimilar input objects x_i and x_j to be modeled by low-dimensional counterparts y_i and y_j that are too far apart because q_{ij} is not very small for two embedded points that are far apart.



t-SNE : Fix Crowding, Illustration

PDF of Gaussian (0,1) and Student-t(1 DOF)



t-SNE: Cost Function

- Cost function

$$C = \sum_i KL(P_i \parallel Q_i) = \sum_{i=1}^n \sum_{j=1}^n p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- **Large** p_{ij} modeled by **small** q_{ij} : **Large** penalty
- **Small** p_{ij} modeled by **large** q_{ij} : **Small** penalty
- **Large** p_{ij} essentially requires **large** q_{ij} to **minimize** C
- t-SNE mainly preserves local similarity structure of the data

Gradient

$$\frac{\partial C}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) (1 + \|y_i - y_j\|^2)^{-1} (y_i - y_j)$$

The gradient of the cost function is :

$$\begin{aligned}
 \frac{dC}{dy_i} &= 4 \sum_{j=1, j \neq i}^n (p_{ij} - q_{ij})(1 + \|y_i - y_j\|^2)^{-1}(y_i - y_j) \\
 &= 4 \sum_{j=1, j \neq i}^n (p_{ij} - q_{ij})q_{ij}Z(y_i - y_j) \\
 &= 4(\sum_{j \neq i} p_{ij}q_{ij}Z(y_i - y_j) - \sum_{j \neq i} q_{ij}^2 Z(y_i - y_j)) \\
 &= 4(F_{\text{attraction}} + F_{\text{repulsion}})
 \end{aligned}$$

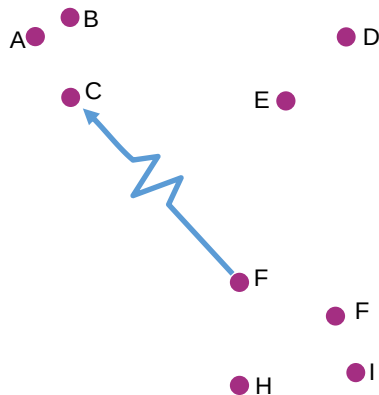
$$\text{where } Z = \sum_{l,s=1, l \neq s}^n (1 + \|y_l - y_s\|^2)^{-1}$$

t-SNE: Physics Analogy N-body system

We can interpret the t-SNE gradient as a simulation of an N-body system

- **Exertion/Compression**

$$(p_{ij} - q_{ij})(1 + \|y_i - y_j\|^2)^{-1}$$

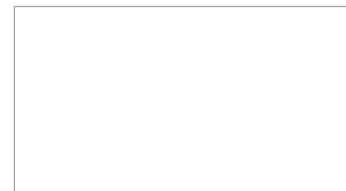
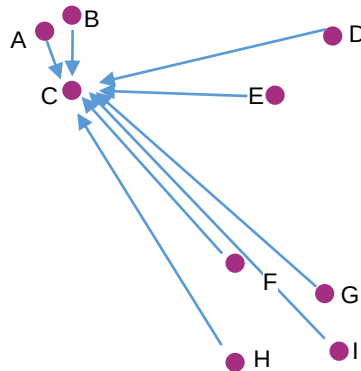


t-SNE: Physics Analogy N-body system

We can interpret the t-SNE gradient as a simulation of an N-body system

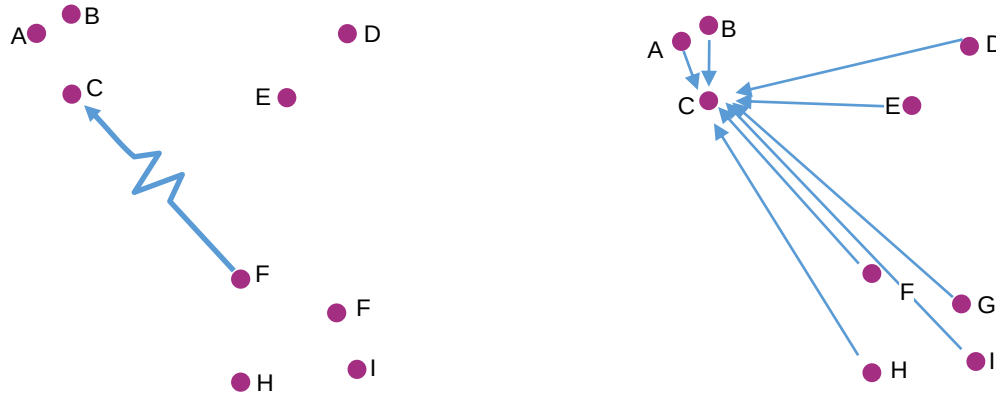
- **N-Body** , summation

$$\frac{\partial \mathcal{C}}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) \left(1 + \|y_i - y_j\|^2 \right)^{-1} (y_i - y_j)$$



t-SNE: Physics Analogy N-body system

$$\frac{\partial \mathcal{C}}{\partial y_i} = 4 \sum_{i \neq j} (p_{ij} - q_{ij}) \frac{y_i - y_j}{1 + \|y_i - y_j\|^2}$$

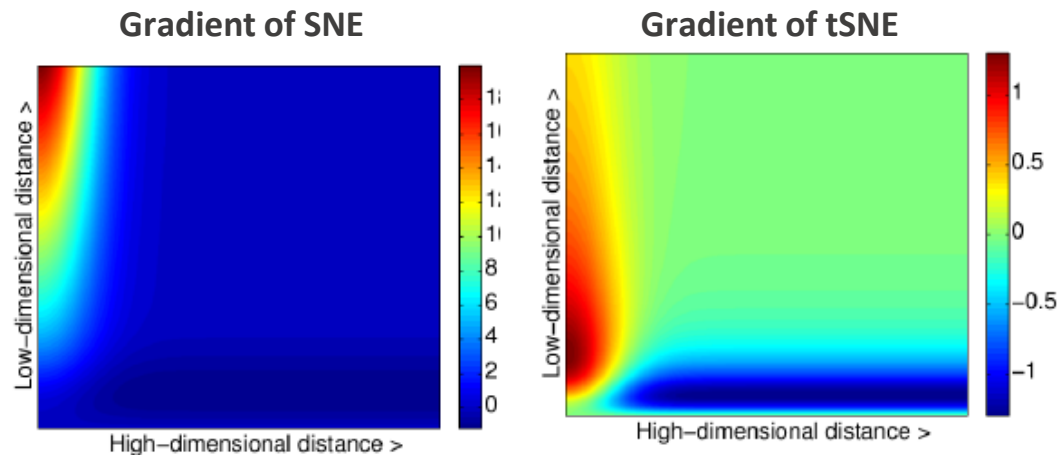


Spring Analogy : $F = -k * (y_i - y_j)$, attraction/repulsion

t-SNE: Gradient Interpretation

X-axis $\rightarrow$ Pairwise Euclidean distance in high-dim (Data Points)

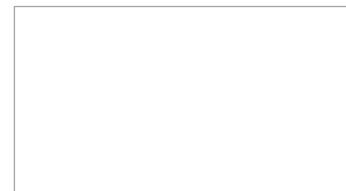
Y-axis $\rightarrow$ Pairwise Euclidean distance in low-dim (Mapped Points)



$$\frac{\partial \mathcal{C}}{\partial y_i} = 4 \sum_{i \neq j} (p_{ij} - q_{ij}) \frac{y_i - y_j}{1 + \|y_i - y_j\|^2}$$

- Positive $\rightarrow$ attraction, negative $\rightarrow$ repulsion
- (Dissimilar Data Points, Similar Mapped Points) $\rightarrow$ strong repulsion
- Repulsions do not go infinity

Gradient of SNE and t-SNE



t-SNE Algorithm

Input : Dataset $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$, Gaussian bandwidths $\sigma_1, \dots, \sigma_n > 0$, step size $h > 0$, momentum parameter $\alpha(t) > 0$, number of round $T \in \mathbb{N}$

1: Compute $\{p_{ij} : i, j \in [n], i \neq j\}$ using

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

2. Initialize $y_1^{(0)}, y_2^{(0)}, \dots, y_n^{(0)}$ i. i. d. From the uniform distribution on $[-0.01, 0.01]^2$

3. **For** $t = 0$ **to** $T - 1$ **do**

4.

$$Z^{(t)} \leftarrow \sum_{i,j \in [n], i \neq j} \left(1 + \|y_i^{(t)} - y_j^{(t)}\|^2\right)^{-1}$$

5.

$$q_{ij}^{(t)} \leftarrow \frac{\left(1 + \|y_i^{(t)} - y_j^{(t)}\|^2\right)^{-1}}{Z^{(t)}}, \forall i, j \in [n], i \neq j$$

6.

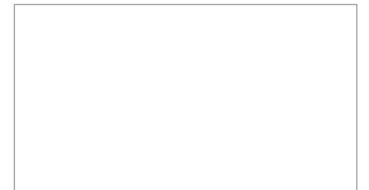
$$y_i^{(t+1)} \leftarrow y_i^{(t)} + \alpha(t) \left(y_i^{(t)} - y_i^{(t-1)}\right) + h \sum_{j \in [n] \setminus \{i\}} \left(p_{ij} - q_{ij}^{(t)}\right) q_{ij}^{(t)} Z^{(t)} \left(y_i^{(t)} - y_j^{(t)}\right), \forall i \in [n]$$

7. **end for**

Output : 2D embedding

$$y^{(T)} = \{y_1^{(T)}, y_2^{(T)}, \dots, y_n^{(T)}\} \subset \mathbb{R}^2$$

- Stochastic Neighbor Embedding (SNE)
- t-SNE
- Applications



MNIST

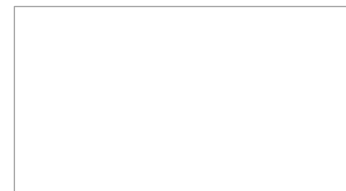
- Randomly selected 6,000 images
- $28 \times 28 = 784$ pixels

Olivetti Faces

- 400 images (10 per individual)
- $92 \times 112 = 10,304$ pixels

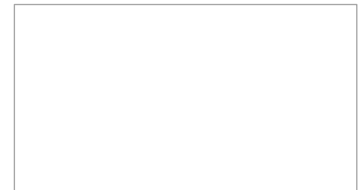
COIL-20

- 20 different objects and 72 equally spaced orientations, yielding a total of 1,440 images
- $32 \times 32 = 1024$ pixels



Technique	Cost function parameters
t-SNE	Perp = 40
Sammon mapping	None
Isomap	K = 12
LLE	K = 12

Table 1 : Cost function parameter settings for the experiments.



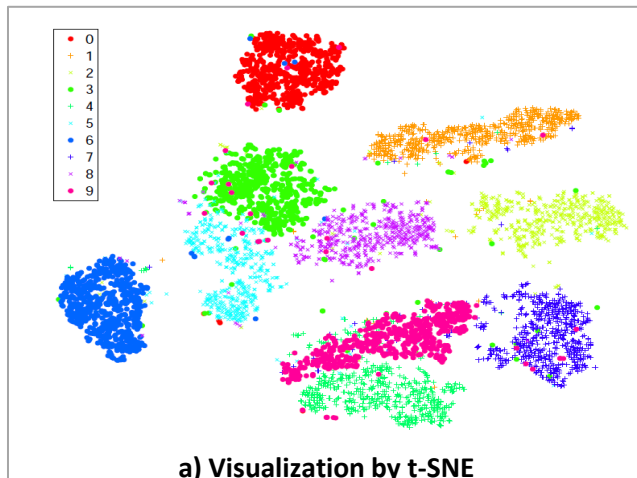
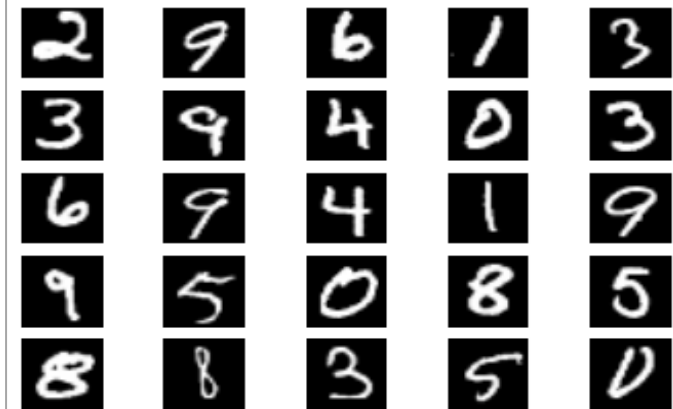
Applications: MNIST

Original high-dimensional
(high-d) space

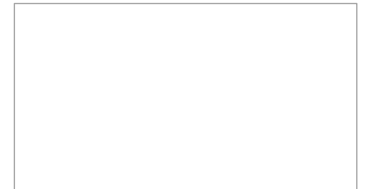
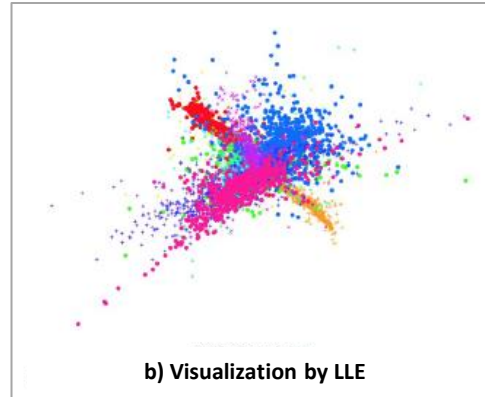
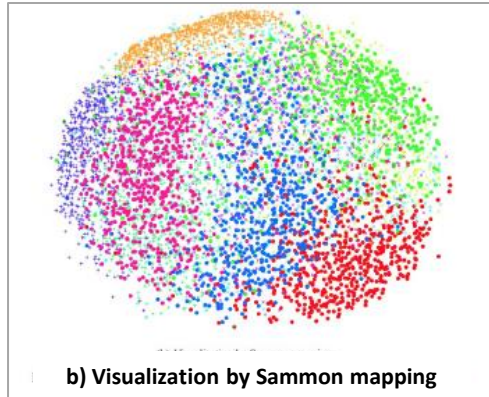
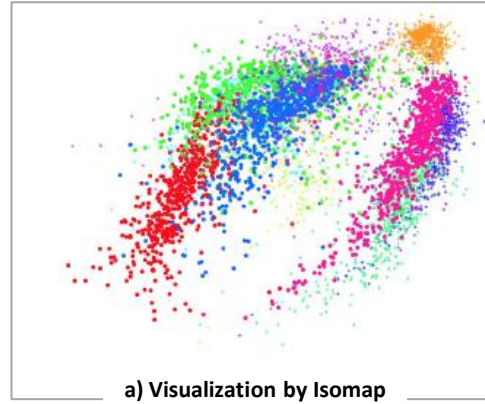
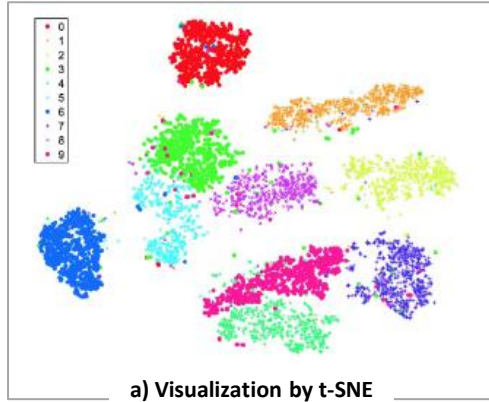


Desired low-dimensional
(low-d) space

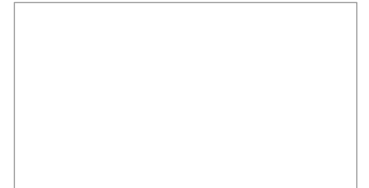
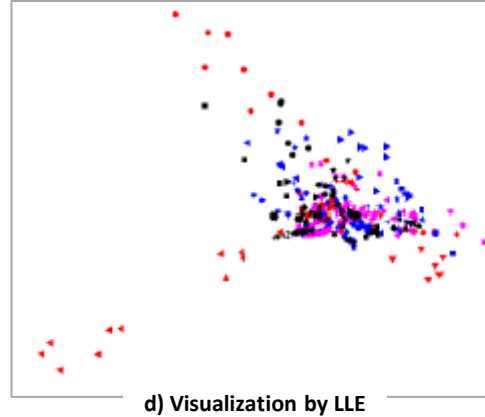
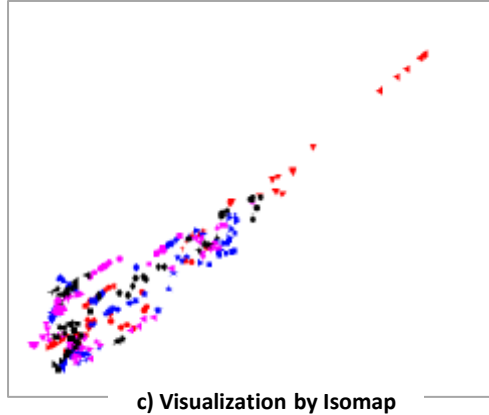
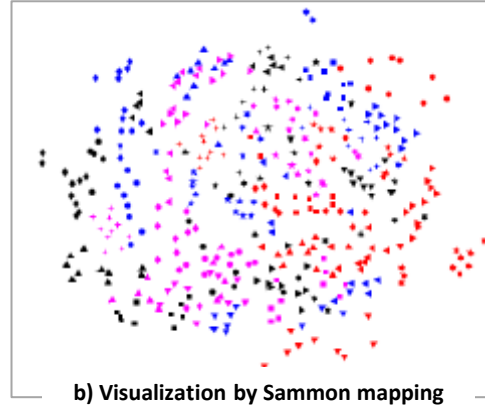
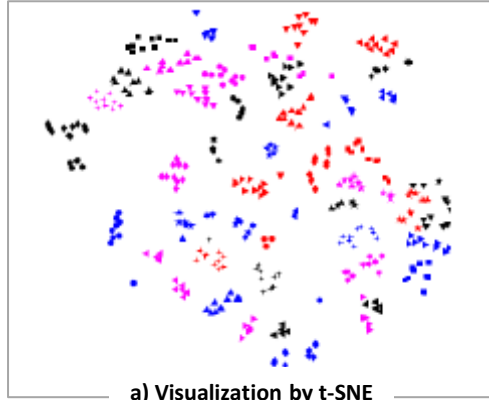
Random Sampling of MNIST



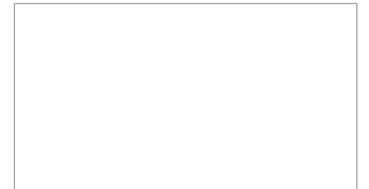
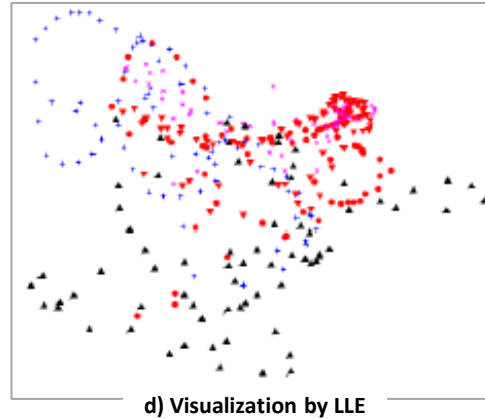
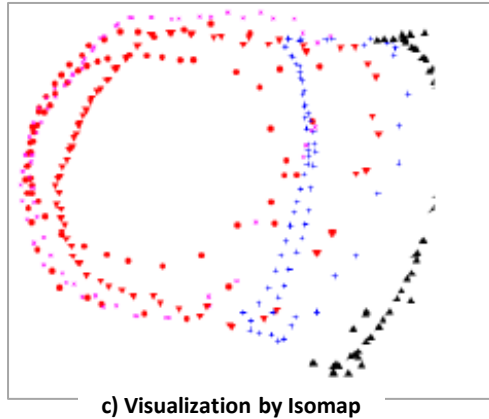
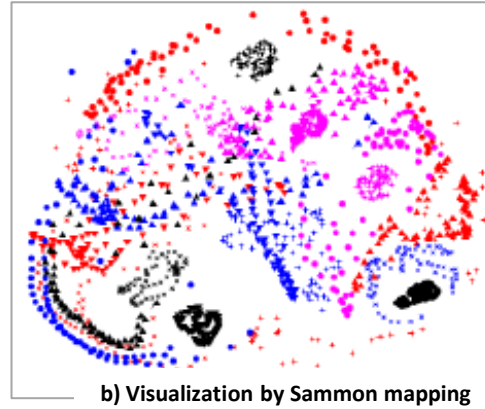
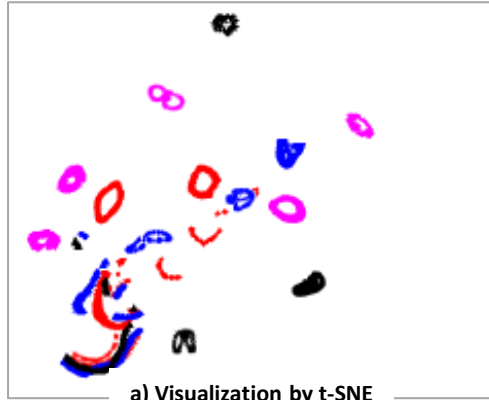
Applications: MNIST



Applications: Olivetti Faces

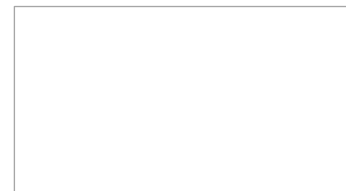


Applications: COIL-20



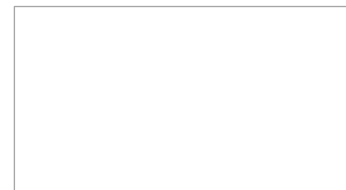
- t-SNE is a non-linear dimensionality reduction method effective for visualizing complex datasets
- t-SNE exposes natural clusters
- t-SNE is robust to outliers
- Original t-SNE data and computational complexity is $O(N^2)$
- Scalable with $O(N \cdot \log(N))$ via Barnes-Hut-SNE (tree-based) algorithm.

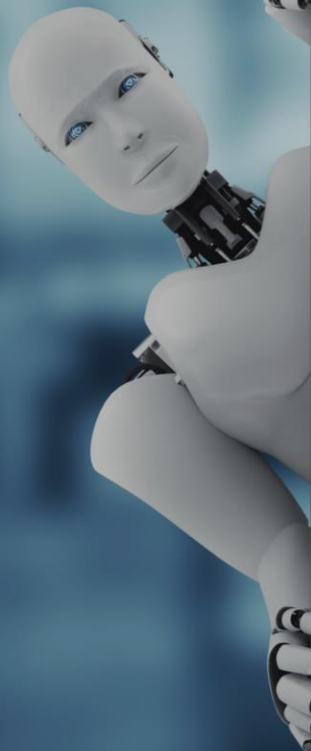
L .v.d. Maaten, "*Barnes-Hut-SNE*", <https://arxiv.org/abs/1301.3342>, 2013



References

- L.v.d. Maaten and G. Hinton, “*Visualizing Data using t-SNE*”, JMLR, 2008.
- L.v.d. Maaten, “*Barnes-Hut-SNE*”, <https://arxiv.org/abs/1301.3342>, 2013
- K. Zhao, “*Visualizing Data using t-SNE*”, <https://www.slideshare.net/ssuserb667a8/visualization-data-using-tsne>, 2014
- D. Khoshid “*Visualizing Data using t-SNE*”, <https://www.slideshare.net/david.kh/visualizing-data-using-tsne>, 2017
- E. Fateya, J. Lucas and E. Andrews, “*CSC 411 Lecture 13:t-SNE*”, https://www.cs.toronto.edu/~jlucas/teaching/csc411/lectures/lec13_handout.pdf
- Z. Zhong, “*t-SNE and its theoretical guarantee*”, http://www.cs.columbia.edu/~verma/classes/uml/lec/uml_lec8_tsne_slides.pdf, 2018





$$\sin^2\theta + \cos^2\theta = 1$$
$$\sec^2\theta = 1 + \tan^2\theta$$
$$\csc^2\theta = 1 + \cot^2\theta$$

$$\sin\frac{\theta}{2} = \pm \sqrt{\frac{1-\cos\theta}{2}}$$
$$\cos\frac{\theta}{2} = \pm \sqrt{\frac{1+\cos\theta}{2}}$$
$$\tan\frac{\theta}{2} = \pm \sqrt{\frac{1-\cos\theta}{1+\cos\theta}}$$

$$\sin\theta = \frac{\text{OPP}}{\text{HYP}}$$
$$\cos\theta = \frac{\text{ADJ}}{\text{HYP}}$$
$$\tan\theta = \frac{\text{OPP}}{\text{ADJ}}$$

$$\csc\theta = \frac{1}{\sin\theta}$$
$$\sec\theta = \frac{1}{\cos\theta}$$
$$\cot\theta = \frac{1}{\tan\theta}$$

$$1 = \sin^2\theta + \cos^2\theta$$
$$1 = \sec^2\theta - \tan^2\theta$$
$$1 = \csc^2\theta - \cot^2\theta$$

$$\sin\frac{\theta}{2} = \pm \sqrt{\frac{1-\cos\theta}{2}}$$
$$\cos\frac{\theta}{2} = \pm \sqrt{\frac{1+\cos\theta}{2}}$$
$$\tan\frac{\theta}{2} = \pm \sqrt{\frac{1-\cos\theta}{1+\cos\theta}}$$

$$\vec{F} = F\cos\theta$$
$$\vec{F}_x = F\cos\theta$$
$$\vec{F}_y = F\sin\theta$$

$$\beta = \frac{I_c}{I_B} = \frac{I_c}{I_E(1-\alpha)}$$

$$\vec{E} \cdot d\vec{l} = -\frac{d\phi_E}{dt}$$
$$\vec{B} \cdot d\vec{l} = \mu_0 I + \mu_0 \epsilon_0 \frac{d\phi_E}{dt}$$

$$\vec{E} \cdot d\vec{l} = (E + dE)h_f - E h_f$$
$$= h_f dE$$

$$\phi_B = BA = B h_f dx$$
$$\frac{d\phi_B}{dt} = h_f dx \frac{dB}{dt}$$

$$\vec{B} \cdot d\vec{l} = -(B + dB)h_2 + B h_2 = -h_2 dB$$
$$\phi_E = EA = E h_2 dx, \frac{d\phi_E}{dt} = h_2 dx \frac{dE}{dt}$$

$$\vec{E} \cdot d\vec{l} = -\frac{d\phi_E}{dt}$$
$$h_f dE = -h_f dx \frac{dB}{dt}$$

$$\frac{dE}{dx} = -\frac{dB}{dt}$$
$$-h_2 dB = \mu_0 \epsilon_0 h_2 dx \frac{dE}{dt}$$

$$\frac{\partial^2 y}{\partial x^2} = \frac{1}{v^2} \frac{\partial^2 y}{\partial t^2}$$
$$y = A \sin(\omega t - kx)$$

$$k = \frac{2\pi}{\lambda}, \frac{1}{v} = \mu_0 \epsilon_0 \frac{\partial^2 B}{\partial t^2} = \mu_0 \epsilon_0 \frac{\partial^2 B}{\partial t^2}$$

$$I_E = I_B + I_C$$
$$I_C = \alpha I_E$$

$$\beta = \frac{I_c}{I_B} = \frac{I_c}{I_E(1-\alpha)}$$

$$P_o = V_o I_o \cos\theta_o$$
$$\cos\theta_o = \frac{P_o}{V_o I_o}$$

$$I_c = I_o \cos\theta_o, I_m = I_o \sin\theta_o$$
$$R_c = \frac{V_o^2}{P_o}, X_m = \frac{V_o}{I_m}$$

$$\rho = \frac{m}{V}$$

$$\frac{\partial \vec{B}}{\partial x} = \mu_0 \epsilon_0 \frac{\partial \vec{E}}{\partial t}$$
$$\frac{\partial \vec{E}}{\partial x^2} = \mu_0 \epsilon_0 \frac{\partial \vec{E}}{\partial t^2}$$

$$V_{BE} = V_{BE}$$
$$I_{BE} = I_{BE}$$

$$I_E = I_B + I_C$$
$$I_C = \alpha I_E$$

$$V_{BE} = V_{BE}$$
$$I_{BE} = I_{BE}$$

$$I_E = I_B + I_C$$
$$I_C = \alpha I_E$$

$$V_{BE} = V_{BE}$$
$$I_{BE} = I_{BE}$$

$$I_E = I_B + I_C$$
$$I_C = \alpha I_E$$

$$A \cap B$$
$$A \cup B$$

$$n(A \cup B) = n(A) + n(B) - n(A \cap B)$$

$$6CO_2 + 12H_2O \xrightarrow{\text{light}} C_6H_{12}O_6 + 6O_2$$
$$3P \rightarrow 3P^+$$

$$Glycine \rightarrow 2P^+ Glycolate + 3P^+ Glycolate$$

$$I-6-DIHEXANOIC ACID$$
$$I-6-DIAMINOHEXANE$$

$$NYLON 6-6$$
$$2AlBr_3 + 3Cl_2 \rightarrow 2AlCl_3$$

$$H-C-C-C(=O)-O-H$$
$$H-C-C-C(=O)-O-H$$

$$H-C-C-C(=O)-O-H$$
$$H-C-C-C(=O)-O-H$$

$$H-C-C-C(=O)-O-H$$
$$H-C-C-C(=O)-O-H$$

$$H-C-C-C(=O)-O-H$$
$$H-C-C-C(=O)-O-H$$

Thank you

Q&A